Chapter 25: All-Pairs Shortest Paths

1) Given a weighted, directed graph $G = (V, E)$ with a weight function $w: E \to R$, we wish to find, for every pair of vertices $u, v \in V$, a shortest path from u to v.

→ We can solve an all-pairs shortest-paths problem by running a single-source shortest-paths algorithm $|V|$ times, once for each vertex as the source. If all edge weights are non-negative, we can use Dijkstra's algorithm. If we use the linear array implementation of the min-priority queue, the running time is $O(V^3 + VE) = O(V^3)$. The binary min-heap implementation of the min-priority queue yields a running time of $O(VE \lg V)$, which is an improvement if the graph is sparse. Alternatively, we can implement the min-priority queue with a Fibonacci heap yielding a running time of $O(V^2 \lg V + VE)$.

If negative weight edges are allowed, we can run the slower BELLMAN-FORD algorithm. The resulting run-time is $O(V^2 E)$, which on a dense graph is $O(V^4)$.

→ In case of single-source algorithms we assume an adjacency-list representation of the graph. But most all-pairs shortest paths algorithms assume an adjacency-matrix representation (except Johnson's algorithm). The input is a $|V| \times |V|$ matrix W representing the edge weights. That is $W = (w_{ij})$, where

$$w_{ij} = \begin{cases} i) 0 & \text{if } i = j \\ ii) \text{the weight of the directed edge } (i,j') & \text{if } i \neq j \text{ and } (i,j') \in E \\ iii) \infty & \text{if } i \neq j' \text{ and } (i,j') \notin E \end{cases}$$

→ The output of "apsp" algorithms is preserved in a tabular form as a $|V| \times |V|$ matrix $D = (d_{ij})$, where d_{ij} holds the weight of a shortest path from vertex i to vertex j. i.e. $d_{ij} = \delta(i,j)$.

→ We compute a predecessor matrix $\Pi = (\pi_{ij})$, where π_{ij} is NIL if either $i = j$ or there is no path from i to j, otherwise π_{ij} is the predecessor of j on some shortest path from i. The subgraph induced by the ith row of the Π matrix should be a shortest-paths tree with root v. For every vertex $i \in V$, we define the "predecessor subgraph" as $G_{\pi,i} = (V_{\pi,i}, E_{\pi,i})$ where:-

$V_{\pi,i} = \{ j \in V : \pi_{ij} \neq NIL \} \cup \{ i \}$, and

$E_{\pi,i} = \{ (\pi_{ij}, j) : j \in V_{\pi,i} - \{ i \} \}$

→ The following procedure prints the shortest-path from vertex i to vertex j:-

PRINT-ALL-PAIRS-SHORTEST-PATH(Π, i, j)

```
1  if i = j
2      then print i
3      else if π_ij = NIL
4              then prints "no path from" i "to" j "exists"
5              else PRINT-ALL-PAIRS-SHORTEST-PATH(
                                                Π, i, π_ij)
6                   print j
```

→ Matrices are denoted by uppercase letters W, L or D, and their individual elements by subscripted lowercase letters such as w_{ij}, l_{ij} or d_{ij}. Matrices having parenthesized superscripts such as $L^{(m)} = (l_{ij}^{(m)})$ indicates iterates.

2) Shortest paths using matrix multiplication

→ This section presents a dynamic programming algorithm for the all-pairs shortest-paths problem on a directed graph.

Structure of a shortest path : Suppose the graph is represented by an adjacency matrix $W = (w_{ij})$. Consider a shortest path from vertex i to vertex j that has at most m edges. If $i = j$, then weight of p is 0 and has no edges. If vertices i and j are distinct, we can decompose the shortest path from i to j into $i \xrightarrow{p'} k \to j$. p' may now contain at most m-1 edges. We have,

$$\delta(i,j) = \delta(i,k) + w(k,j)$$

A recursive solution to the apsp problem:

Let $l_{ij}^{(m)}$ be the "minimum" weight of "any" path from vertex i to vertex j that contains at most m edges. Thus,

$$l_{ij}^{(0)} = \begin{cases} 0 & , \text{if } i = j \\ \infty & , \text{if } i \neq j \end{cases}$$

We recursively define $l_{ij}^{(m)}$ as :-

$$l_{ij}^{(m)} = \min\left(l_{ij}^{(m-1)} , \min_{1 \le k \le n} \{ l_{ik}^{(m-1)} + w_{kj} \} \right)$$

(we consider all possible predecessors k of j).

If the graph does not contain any negative weight cycle, then for every vertex j reachable from i, the shortest path from i to j is simple and contains at most (n-1) edges. A path from vertex i to vertex j with more than (n-1) edges cannot have more weight than a shortest path from i to j. Thus :—

$$\delta(i,j) = l_{ij}^{(n-1)} = l_{ij}^{(n)} = l_{ij}^{(n+1)} \ldots\ldots$$

Computing the shortest-path weights bottom up:

We compute the series of matrices $L^{(1)}, L^{(2)}, L^{(3)}$ $\ldots\ldots L^{(n-1)}$. The final matrix $L^{(n-1)}$ contains the actual shortest-path weights.

The matrix $L^{(x)}$ contains the shortest-path weights from every vertex i to every vertex j, that can be at most x edges long. If 2 vertices i and j are more than x edges apart at the beginning, then $l_{ij}^{(x)} = \infty$. Hence, note that $L^{(1)} = W$.

The procedure EXTEND-SHORTEST-PATHS takes the matrices $L^{(m-1)}$ and W as input and returns $L^{(m)}$ as output. In other words it extends the shortest paths computed so far by one more edge.

EXTEND-SHORTEST-PATHS (L, W)

```
1  n ← rows[L]
2  let L' = (l'ij) be an n × n matrix
3  for i ← 1 to n
```

```
4       do for j ← 1 to n
5           do l'ij ← ∞
6               for K ← 1 to n
7                   do l'ij ← min(l'ij, liK + WKj)
8   return L'
```

$$l_{ij}^{(m)} = \min\left(l_{ij}^{(m-1)}, \min_{1 \le K \le n}\left\{l_{iK}^{(m-1)} + W_{Kj}\right\}\right)$$

$$= \min_{1 \le K \le n}\left\{l_{iK}^{(m-1)} + W_{Kj}\right\}$$

[$\because K = j$ includes the 1st case and $W_{Kj} = W_{jj} = 0$]

$L^{(m)}$ can be computed by the multiplication of $L^{(m-1)}$ with W by noting that $C = A \cdot B$ can be computed as $c_{ij} = \sum_{K=1}^{n} a_{iK} \cdot b_{Kj}$. We make the following substitutions :

$l^{(m-1)} \rightarrow a$	We make these changes to
$W \rightarrow b$	EXTEND-SHORTEST-PATHS and
$l^{(m)} \rightarrow c$	replace ∞ (the identity for min)
$\min \rightarrow +$	by 0 (the identity for +), we
$+ \rightarrow \cdot$	obtain the following $\Theta(n^3)$ procedure:-

EXTEND-SHORTEST-PATHS (L, W)

```
1   n ← rows[L]
2   let L' = (l'ij) be an n×n matrix
3   for i ← 1 to n
4       do for j ← 1 to n
```

```
5        do l_ij ← 0
6           for k ← 1 to n
7              do l_ij ← l_ij + l_ik · w_kj
8  return L'
```

Thus,

$$L^{(1)} = W$$
$$L^{(2)} = L^{(1)} \cdot W = W^2$$
$$L^{(3)} = L^{(2)} \cdot W = W^3$$
$$\vdots$$
$$L^{(n-1)} = L^{(n-2)} \cdot W = W^{(n-1)}$$

The following procedure computes $L^{(n-1)}$ in $\Theta(n^4)$ time:-

SLOW-ALL-PAIRS-SHORTEST-PATHS(W)

```
1  n ← rows[W]
2  L^(1) ← W
3  for m ← 2 to n-1
4     do L^(m) ← EXTEND-SHORTEST-PATHS(L^(m-1), W)
5  return L^(n-1)
```

Improving the running time:

We can compute $L^{(n-1)}$ with only $\lceil \lg(n-1) \rceil$ matrix products by computing the sequence :-

$$L^{(1)} = W$$

3) The Floyd-Warshall algorithm:

The floyd-warshall algorithm is a dynamic-programming formulation to solve the apsp problem on a directed graph. Negative-weight edges may be present, but as before we assume that there are no negative-weight cycles.

The structure of a shortest path

An intermediate vertex of a simple path $p = \langle v_1, v_2, v_3 \ldots v_e \rangle$ is any vertex of p other than v_1 or v_e, i.e. any vertex in the set $\{v_2, v_3 \ldots v_{e-1}\}$.

Suppose the vertices of a graph G are numbered as $V = \{1, 2, 3 \ldots n\}$ and consider a subset of vertices $\{1, 2, 3 \ldots k\}$ for some value of $k \leq n$. For any pair of vertices $i, j \in V$, consider all paths from i to j whose intermediate vertices are all drawn from $\{1, 2 \ldots k\}$ and let p be the minimum-weight path among them. (Path p is simple).

(i) If k is an intermediate vertex of path p, then we break p down into $i \overset{p_1}{\rightsquigarrow} k \overset{p_2}{\rightsquigarrow} j$. The optimal substructure of shortest paths property states that p_1 must be a shortest path from i to k with all intermediate vertices in the set $\{1, 2 \ldots k-1\}$. Similarly, p_2 must be a shortest path from k to j with all intermediate vertices in the set $\{1, 2 \ldots k-1\}$.

(ii) If k is not an intermediate vertex of path p, then all intermediate vertices of path p are in the set $\{1, 2, \ldots k-1\}$. In other words, a shortest path

$$L^{(2)} = W^2 = W \cdot W$$

$$L^{(4)} = W^4 = W^2 \cdot W^2$$

$$L^{(8)} = W^8 = W^4 \cdot W^4$$

$$\vdots$$

$$L^{(2^{\lceil \lg(n-1) \rceil})} = W^{2^{\lceil \lg(n-1) \rceil}} = W^{2^{\lceil \lg(n-1) \rceil - 1}} \cdot W^{2^{\lceil \lg(n-1) \rceil - 1}}$$

Since $2^{\lceil \lg(n-1) \rceil} \geq n-1$ and because $L^{(m)} = L^{n-1}$ for $m \geq n-1$ we have $L^{(2^{\lceil \lg(n-1) \rceil})} = L^{(n-1)}$.

The following $\Theta(n^3 \lg n)$ procedure computes the above sequence by "repeated squaring":-

FASTER_ALL_PAIRS_SHORTEST_PATHS(W)

```
1  n ← rows[W]
2  L^(1) ← W
3  m ← 1
4  while m < n-1
5        do L^(2m) ← EXTEND_SHORTEST_PATHS(L^(m), L^(m))
6           m ← 2m
7  return L^(m)
```

Exit from the while loop occurs when $n-1 \leq 2m < 2n-2$ or, $\lg(n-1) \leq m \leq \lg 2(n-1)$.

~~and then you let~~

from vertex i to vertex j with all intermediate vertices in the set $\{1, 2 \ldots k-1\}$ is also a shortest-path from i to j with all intermediate vertices in the set $\{1, 2 \ldots k\}$.

A recursive solution to the apsp problem:

Let $d_{ij}^{(k)}$ be the weight of a shortest path from vertex i to vertex j having all intermediate vertices in the set $\{1, 2 \ldots k\}$. When $k = 0$, you cannot choose any intermediate vertex with no. > than 0, which means there cannot be intermediate vertices at all (since vertex no.s start at 1). Such a path has at most 1 edge. Hence, $d_{ij}^{(0)} = w_{ij}$.

$$d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{, if } k = 0 \\ \min\left(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\right) & \text{, if } k \geq 1 \end{cases}$$

Because for any path, all intermediate vertices are in the set $\{1, 2, \ldots n\}$, the matrix $D^{(n)} = (d_{ij}^{(n)})$ gives the final answer: $d_{ij}^{(n)} = \delta(i, j)$ for all $i, j \in V$.

Computing the shortest-path weights bottom up:

FLOYD-WARSHALL (W)

```
1   n ← rows[W]
2   D^0 ← W
3   for k ← 1 to n
4       do for i ← 1 to n
5           do for j ← 1 to n
```

6 do $d_{ij}^{(k)} \leftarrow \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$

7 return $D^{(n)}$

→ Since each execution of line 6 takes $O(1)$ time, the algorithm runs in time $\Theta(n^3)$. The code is tight, with no elaborate data structures, and so the constant hidden in the Θ-notation is small.

Constructing a shortest path:

We compute a sequence of matrices $\Pi^{(0)}, \Pi^{(1)}, \Pi^{(2)} \ldots \Pi^{(n)}$ where $\pi_{ij}^{(k)}$ is defined to be the predecessor of vertex j on a shortest path from vertex i with all intermediate vertices in the set $\{1, 2 \ldots k\}$.

When $k = 0$, there are no intermediate vertices at all. Hence,

$$\pi_{ij}^{(0)} = \begin{cases} i & \text{, if } i \neq j \text{ and } w_{ij} < \infty \\ \text{NIL} & \text{, if either } i = j \text{ or } w_{ij} = \infty \end{cases}$$

In other words, the matrix $\Pi^{(0)}$ is computed from W by letting the entry $\pi_{ij}^{(0)}$ be i if there is an edge $(i,j) \in E$ i.e. $w_{ij} < \infty$ and NIL otherwise.

$$\pi_{ij}^{(k)} = \begin{cases} \pi_{ij}^{(k-1)} & \text{, if } d_{ij}^{(k-1)} \leq d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \\ \pi_{kj}^{(k-1)} & \text{, if } d_{ij}^{(k-1)} > d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \end{cases}$$

Computation of the $\Pi^{(k)}$ matrix can be incorporated into the FLOYD-WARSHALL algorithm.

4) Transitive closure of a directed graph:

Given a directed graph $G = (V, E)$ with vertex set $V = \{1, 2, \ldots, n\}$, we may wish to find out whether there is a path in G from i to j for all vertex pairs $i, j \in V$.

The transitive closure of G is defined as the graph $G^* = (V, E^*)$, where

$$E^* = \{(i,j) : \text{there is a path from vertex } i \text{ to vertex } j \text{ in } G\}$$

→ One way to compute the transitive closure of a graph in $\Theta(n^3)$ time is to assign a weight of 1 to each edge of E and run the Floyd-Warshall algorithm. If there is a path from vertex i to vertex j, we get $d_{ij} < n$. Otherwise, we get $d_{ij} = \infty$. There is a similar way to compute the transitive-closure of G in $\Theta(n^3)$ time that can save time and space in practice.

→ We define $t_{ij}^{(k)}$ to be 1 if there exists a path in graph G from vertex i to vertex j with all intermediate vertices in the set $\{1, 2, \ldots, k\}$ and 0 otherwise. We put the edge (i,j) into E^* if and only if $t_{ij}^{(n)} = 1$.

$$t_{ij}^{(0)} = \begin{cases} 1, & \text{if } i = j \text{ or } (i,j) \in E \\ 0, & \text{if } i \neq j \text{ and } (i,j) \notin E \end{cases}$$

and for $k \geq 1$, we have:—

$$t_{ij}^{(K)} = t_{ij}^{(K-1)} \vee \left(t_{iK}^{(K-1)} \wedge t_{Kj}^{(K-1)} \right)$$

The following procedure computes $T^{(K)} = (t_{ij}^{(K)})$ in order of increasing K :-

TRANSITIVE_CLOSURE(G)

```
1   n ← |V[G]|
2   for i ← 1 to n
3       do for j ← 1 to n
4           do if i=j or (i,j) ∈ E[G]
5               then t_ij^(0) ← 1
6               else t_ij^(0) ← 0
7   for K ← 1 to n
8       do for i ← 1 to n
9           do for j ← 1 to n
10              do t_ij^(K) ← t_ij^(K-1) ∨ (t_iK^(K-1) ∧ t_Kj^(K-1))
11  return T^(n)
```

→ The TRANSITIVE_CLOSURE procedure runs in $\Theta(n^3)$ time. On some computers, logical-operations on single-bit values execute faster than arithmetic operations on integer words of data. Moreover, because the direct transitive-closure algorithm uses only boolean values rather than integer

values, its space requirement is less than that of Floyd-Warshall's algorithm by a factor that corresponds to the size of a word of computer memory.

5) Johnson's algorithm for sparse graphs:

→ Johnson's algorithm finds shortest paths between all pairs in $O(V^2 \lg V + VE)$ time. For sparse graphs, it is asymptotically better than either repeated squaring of matrices or the Floyd-Warshall algorithm. The algorithm either returns a matrix of shortest-path weights for all pairs of vertices or reports that the input graph contains a negative-weight cycle.

→ Johnson's algorithm uses both Dijkstra's algorithm and Bellman-Ford algorithm as subroutines.
Johnson's algorithm uses the technique of reweighting. If graph G has no negative-weight cycles (but may have negative-weight edges) we compute a new set of nonnegative edge weights.

The new set of edge weights $\hat{w}$ must satisfy two important properties:-

1. For all pairs of vertices $u, v \in V$, a path p is a shortest path from u to v using weight function w if and only if p is also a shortest path from u to v using weight function $\hat{w}$.
2. For all edges (u, v), the new weight $\hat{w}(u, v)$ is non-negative.

→ Preserving shortest paths by reweighting

Suppose we use δ to denote shortest-path weights using w and $\hat{\delta}$ to denote shortest-path weights derived using $\hat{w}$.

Lemma (Reweighting does not change shortest paths)

Given a weighted, directed graph $G = (V, E)$ with weight function $w: E \to R$, let $h: V \to R$ be any function mapping vertices to real numbers. For each edge $(u, v) \in E$, define

$$\hat{w}(u, v) = w(u, v) + h(u) - h(v)$$

Let $p = \langle v_0, v_1, v_2, \ldots, v_k \rangle$ be any path from vertex v_0 to vertex v_k. Then p is a shortest path from v_0 to v_k with weight function w if and only if p is a shortest path using $\hat{w}$. That is, $w(p) = \delta(v_0, v_k)$ if and only if $\hat{w}(p) = \hat{\delta}(v_0, v_k)$.

Also, G has a negative-weight cycle using weight function w if and only if G has a negative-weight cycle using weight-function $\hat{w}$.

Proof:

$$\hat{w}(p) = \sum_{i=1}^{k} \hat{w}(v_{i-1}, v_i)$$

$$= \sum_{i=1}^{k} \{ w(v_{i-1}, v_i) + h(v_{i-1}) - h(v_i) \}$$

$$= \sum_{i=1}^{k} w(v_{i-1}, v_i) + h(v_0) - h(v_k)$$

{because the sum telescopes}

$= w(p) + h(v_0) - h(v_k)$

Therefore, any path p from v_0 to v_k has $\hat{w}(p) = w(p) + h(v_0) - h(v_k)$. If one path from v_0 to v_k is shorter than another using weight-function w, then it also shorter using $\hat{w}$. Hence, $w(p) = \delta(v_0, v_k)$ if and only if $\hat{w}(p) = \hat{\delta}(v_0, v_k)$.

Next, consider any cycle $c = \langle v_0, v_1, v_2 \ldots v_k \rangle$ where $v_0 = v_k$.

$$\hat{w}(c) = w(c) + h(v_0) - h(v_k)$$
$$= w(c)$$

If $w(c) < 0$, then $\hat{w}(c) < 0$ implying that G has a negative weight cycle using weight-function w if and only if G has a negative-weight cycle using weight-function $\hat{w}$.

→ Producing non-negative weights by reweighting

Given a weighted directed graph $G = (V, E)$ with weight function $w: E \to R$, we make a new graph $G' = (V', E')$ where $V' = V \cup \{s\}$ for some new vertex $s \notin V$ and $E' = E \cup \{(s, v) : v \in V\}$. We extend the weight-function w so that $w(s, v) = 0$ for all $v \in V$.

Note that because s has no edges that enter it, no shortest paths in G', other than those with source s, contain s. Moreover, G' has no negative-weight cycles if and only if G has no negative-weight cycles.

We define $h(v) = \delta(s, v)$ for all $v \in V'$.
By the triangle inequality we have :

$h(v) \leq h(u) + w(u,v)$ for all edges $(u,v) \in E'$.

$\Rightarrow \quad w(u,v) + h(u) - h(v) \geq 0$

Thus, $\hat{w}(u,v) = w(u,v) + h(u) - h(v) \geq 0$

Computing all-pairs shortest paths:

Johnson's algorithm assumes that the edges are stored in adjacency lists. It returns the usual $|V| \times |V|$ matrix $D = (d_{ij})$, where $d_{ij} = \delta(i,j)$, or it reports that the input graph contains a negative-weight cycle. We assume that the vertices are renumbered from 1 to $|V|$.

JOHNSON(G)

1 compute G', where $V[G'] = V[G] \cup \{s\}$,
$E[G'] = E[G] \cup \{(s,v) : v \in V[G]\}$ and $w(s,v) = 0$
for all $v \in V[G]$

2 if BELLMAN_FORD(G', w, s) = FALSE

3 then print "the I/P graph contains a -ve-wt cycle"

4 else for each vertex $v \in V[G']$

5 do set $h(v)$ to the value $\delta(s,v)$
computed by the BELLMAN_FORD algorithm

6 for each edge $(u,v) \in E[G']$

7 do $\hat{w}(u,v) \leftarrow w(u,v) + h(u) - h(v)$

8 for each vertex $u \in V[G]$

9 do run DIJKSTRA$(G, \hat{w}, u)$ to
compute $\delta(u,v)$ for all $v \in V[G]$

10 for each vertex $v \in V[G]$

11 do $d_{uv} \leftarrow \hat{\delta}(u,v) + h(v) - h(u)$

12 return D

→ If the min-priority queue in Dijkstra's algorithm is implemented by a Fibonacci heap, the running time of Johnson's algorithm is $O(V^2 \lg V + VE)$. The simpler binary min-heap implementation yields a running time of $O(VE \lg V)$, which is still asymptotically faster than the Floyd-Warshall algorithm if the graph is sparse.